A Project Report on

Supplier Bid Optimization Engine

Prepare By
Suyash S. Kadu

INDEX

1. Project Introduction:-

1.1. Project Overview

The Supplier Bid Optimization Engine (SBOE) is an enterprise-grade, Coupa-inspired strategic sourcing tool designed to evaluate raw material suppliers. The engine specifically evaluates vendors providing commodities like steel, lubricants, and bearings, and recommends the optimal way to split an order. The core philosophy driving the SBOE is that in global manufacturing, the lowest unit price is rarely the cheapest final cost.

1.2. The Business Context & Problem

Traditional purchasing often focuses strictly on the initial sticker price. This approach ignores critical hidden risks within the supply chain. For example, the cheapest supplier might have a 3% historical defect rate or take 45 days to deliver. These hidden variables drastically increase the final operational cost, introducing massive financial bleed that basic price comparisons fail to catch.

1.3. The Strategic Solution

To solve this challenge, the SBOE acts as a "financial lie-detector" for supplier bids by forcing procurement decisions to be based on Total Cost of Ownership (TCO). The mathematical engine calculates the true bottom line by explicitly penalizing vendors for hidden costs. The baseline TCO evaluation factors in:

- The initial unit price.
- Flat shipping and logistics costs.
- Historical defect rates.
- Lead times and delivery delays.

1.4. Business Value & Technical Execution

By shifting the focus from unit price to true bottom-line TCO, the SBOE translates complex supply chain variables into actionable financial decisions. It is specifically designed to target the needs of enterprise manufacturing companies like SKF and features high relevance to Indian manufacturing hubs like Pune and Chennai. Key capabilities include:

- Interactive Scenario Modeling: It utilizes an interactive Streamlit dashboard that empowers procurement managers to run dynamic "what-if" scenarios.
- High-Performance Processing: The logic engine is built with Python and Pandas, utilizing vectorization to ensure lightning-fast, highly scalable calculations without relying on slow for-loops.

2. System & Competency Requirements:-

2.1. Technical Dependencies

- Proficiency in Python programming.
- Familiarity with Pandas for data manipulation and vectorization.
- Basic understanding of Streamlit for frontend UI development.

2.2. Data Architecture & Schema Requirements

- A structured dataset featuring realistic supplier profiles, item definitions, and historical bid metrics.
- Data structured relationally, ideal for databases like Supabase (PostgreSQL).
- Contextual data relevance to realistic manufacturing environments, such as Indian manufacturing hubs like Pune and Chennai.

2.3. Analytical & Domain Competencies

- A foundational understanding of supply chain metrics and strategic sourcing.
- Deep comprehension of Total Cost of Ownership (TCO) mathematical models and supply chain risk mitigation.
- The business acumen to translate complex supply chain variables into actionable financial decisions.

3. Objectives:-

3.1. Primary Objective

To architect and deploy a scalable, interactive procurement tool that forces purchasing decisions to be based on the true bottom-line Total Cost of Ownership (TCO), rather than superficial initial unit prices.

3.2. Scope of Work

- **Algorithmic Engine:** Develop a high-performance mathematical engine using Python and Pandas to calculate TCO by penalizing vendors for hidden supply chain risks, specifically historical defect rates and lead time delays.
- **Interactive Interface:** Design an enterprise-grade user interface using Streamlit, styled with SKF Corporate Blue, to enable dynamic "What-If" scenario modeling for procurement managers.
- **Data Architecture:** Structure and manage a realistic relational dataset (simulating Indian manufacturing hubs like Pune and Chennai) containing supplier profiles, item definitions, and historical bid metrics.
- **Cloud Deployment:** Deploy the application to a cloud environment (e.g., Hugging Face Spaces or Render) to demonstrate production readiness and CI/CD capabilities.

3.3. Out of Scope

Integration with live enterprise ERP systems (like SAP or Oracle) or execution of actual financial transactions. This project serves as an analytical recommendation engine, not a purchasing execution platform.

4. Problem Statement:-

- Traditional purchasing often focuses strictly on the initial sticker price. This approach ignores critical hidden risks within the supply chain.
- By evaluating bids purely on unit cost, procurement managers inadvertently introduce massive financial bleed that basic price comparisons fail to catch. For example, the cheapest supplier might have a 3% historical defect rate or take 45 days to deliver.
- These hidden variables drastically increase the final operational cost. In global manufacturing, the lowest unit price is rarely the cheapest final cost.

5. Technology Architecture & Tech Stack:-

The Supplier Bid Optimization Engine (SBOE) utilizes a consolidated, Python-centric technology stack designed for rapid development and high-performance data processing.

- **Coding Language: Python**. The core language driving both the analytical backend and the frontend framework.

- **Data Processing Engine: Pandas & NumPy**. Utilized for high-performance vectorization to calculate Total Cost of Ownership (TCO) metrics across all rows simultaneously. This completely avoids slow for-loops to ensure lightning-fast execution and enterprise scalability.

- **Frontend & Backend Framework: Streamlit**. A unified framework that acts as both the web server and the interactive user interface. It empowers procurement managers to run dynamic "what-if" scenarios using slider inputs.

- **Database Architecture: Supabase (PostgreSQL)**. A cloud-based relational database used to handle structured data across core tables: Suppliers, Items, and Bids.

- **Cloud Deployment & CI/CD: Render / Streamlit Community Cloud**. Cloud platforms utilized for live, public-facing web service deployment with continuous integration/continuous deployment capabilities directly from GitHub.

6. Phase 1: Strategic Planning & Architecture Design:-

6.1. The Core Objective

The primary objective of Phase 1 is to establish the core mathematical backend the Pandas Optimization Engine before introducing any user interface or database connections.

6.2. The Strategy

The strategic goal is to build an analytical engine that acts as a financial lie-detector. It ingests raw supplier data, normalizes it, and calculates the true bottom line by explicitly penalizing vendors for hidden costs like historical defect rates and lead time delays.

6.3. Execution Focus

By focusing entirely on the Python logic first, the business logic remains successfully decoupled from the eventual Streamlit UI. This ensures that the engine is highly scalable and can be tested locally using a mocked CSV dataset (e.g., a scenario of 5 suppliers bidding on 10,000 bearings).

7. Phase 1: Implementation & Deployment:-

Phase 1 executes the core analytical engine in Python. It involves breaking down the execution into sub-steps, from establishing the mathematical formula to testing locally and deploying to the cloud.

7.1. Core Business Logic & Algorithmic Ruleset

The engine normalizes data, so every supplier is judged on the exact same criteria. It calculates the true bottom line by factoring in unit price, shipping costs, historical defect rates, and lead times.

7.2. Total Cost of Ownership (TCO) Mathematical Model

The baseline TCO evaluation takes the unit price and stacks financial "penalties" on top of it. The exact formula implemented is:

$$TCO = (P \cdot Q) + S + (D \cdot Q \cdot C_d) + (L \cdot C_l)$$

7.3. Data Ingestion & Relational Structuring

For Phase 1, the data is consolidated into a single, comprehensive flat table (CSV) to streamline ingestion into the Pandas optimization engine. Rather than using multiple relational tables, this unified dataset encompasses all requisite variables in one place. It seamlessly combines overarching supplier profiles (e.g., Reliability Score, Region, Max Capacity), item definitions (e.g., Baseline Cost, SKU, Commodity), and the dynamic

transactional bid inputs (e.g., Unit Price, Lead Time, Defect Rate) required for the algorithm.

supplier_id	supplier_name	region	reliability_score	payment_terms	max_capacity	item_id	sku	description	commodity_category
SUP-017	APL Apollo Tubes Ltd	Uttar Pradesh	85	Net 45	5707	ITM-030	SKU-OILSEAL-TC	TC Rotary Oil Seal 50×70×10	Seals & Gaskets
SUP-025	Tube Investments of India	Tamil Nadu	82	Advance 50% Net 30	7461	ITM-032	SKU-FASTNR-M12	M12×50 Hex Bolt 8.8 Zinc (100pcs)	Fasteners
SUP-034	Grindwell Norton Ltd	Maharashtra	65	Net 30	10533	ITM-022	SKU-COMPOL-VG100	Compressor Oil VG100 50L	Lubricants & Fluids
SUP-051	Bosch Ltd (India)	Karnataka	87	Net 15	308	ITM-021	SKU-CUTFL-SOL	Soluble Cutting Fluid Concentrate 25L	Lubricants & Fluids
SUP-030	Lakshmi Machine Works	Tamil Nadu	83	Cash Against Documents	5763	ITM-025	SKU-ENDML-TIN12	TiN Coated End Mill 12mm 4-Flute	Abrasives & Tooling
SUP-048	NOCIL Ltd	Maharashtra	65	Advance 30% Net 60	413	ITM-024	SKU-DRLBIT-TCR10	Tungsten Carbide Drill Bit 10mm	Abrasives & Tooling
SUP-057	Finolex Industries Ltd	Maharashtra	71	Net 45	3106	ITM-026	SKU-ABPAPER-P120	Abrasive Sheet P120 230×280	Abrasives & Tooling
SUP-003	Mahindra CIE Automotive	Maharashtra	89	LC 60 Days at Sight	13756	ITM-027	SKU-CBDWHL-D126	CBN Grinding Wheel D126 100×10	Abrasives & Tooling
SUP-028	Rane Holdings Ltd	Tamil Nadu	75	LC 90 Days	3016	ITM-029	SKU-GASK-PTFE3	PTFE Sheet Gasket 3mm 500×500	Seals & Gaskets
SUP-048	NOCIL Ltd	Maharashtra	65	Advance 30% Net 60	413	ITM-034	SKU-FASTNR-STUD	M16 Stud Bolt B7/2H Set (10pcs)	Fasteners
SUP-003	Mahindra CIE Automotive	Maharashtra	89	LC 60 Days at Sight	13756	ITM-005	SKU-STLPIP-GI50	Galvanised Iron Pipe DN50 6m	Ferrous Metals
SUP-007	Prakash Industries Ltd	Chhattisgarh	63	Net 30	7040	ITM-011	SKU-ZNCING-SHG	Zinc SHG Ingot 25kg	Non-Ferrous Metals
SUP-030	Lakshmi Machine Works	Tamil Nadu	83	Cash Against Documents	5763	ITM-039	SKU-VBELT-B68	Classical V-Belt B68	Conveyor & Drive
SUP-051	Bosch Ltd (India)	Karnataka	87	Net 15	308	ITM-006	SKU-STLANG-MS	MS Equal Angle 50×50×6	Ferrous Metals

unit_weight_kg	moq	baseline_cost_inr	unit_price_inr	lead_time_days	defect_rate_pct	qty_ordered	logistics_cost_inr	cost_per_defect_inr	penalty_per_day_inr
0.05	500	220	209.55	53	4.8987	1104	1201.49	332.84	696.97
1.1	500	380	366.57	13	3.031	532	4558.1	355.87	156.86
47	20	7600	7620.57	26	4.784	20	1567.38	12206.6	208.65
24	50	4200	4258.57	7	2.4926	87	16908.33	8239.29	184.33
0.08	200	620	544.49	8	4.4618	210	1417.62	574.47	206.34
0.05	500	280	282.96	22	4.9249	500	1157.66	606.25	42.44
0.02	2000	42	39.77	9	4.3659	2000	2939.7	73.45	107.9
1.6	50	8400	9051.39	15	4.9742	50	1016.86	8190.2	839.61
1.2	50	1450	1366.98	37	1.7552	50	1942.48	1990.45	60.47
1.8	100	640	631.55	31	2.0979	103	2460.61	1085.9	63.28
18	50	3600	3747.94	46	5	50	2899.63	7985.74	163.41
25	200	24500	26374.56	43	7.8366	200	34788.37	15491.95	7594.04
0.3	200	240	240.54	27	4.7621	351	2898.85	465.39	71.75
45	25	8200	8444.14	44	2.7499	52	23147.1	16725.81	374.88

7.4. Initial Research & Development (R&D) via Vectorized Operations

Initial R&D was conducted in a Jupyter Notebook to test the math without UI overhead. To ensure lightning-fast execution and enterprise scalability, Pandas vectorization was utilized to calculate metrics across all rows simultaneously, avoiding slow for-loops entirely.

```
[1]: import pandas as pd

[2]: def calculate_tco(df, target_item, quantity, cost_per_defect, penalty_per_day):

        bids = df[df["description"] == target_item].copy()

        bids["Base_Cost"] = bids["unit_price_inr"] * quantity

        bids["Defect_Cost"] = (bids["defect_rate_pct"] / 100) * quantity * cost_per_defect

        bids["Lead_Time_Penalty"] = bids["lead_time_days"] * penalty_per_day

        bids["TCO"] = (
            bids["Base_Cost"] +
            bids["logistics_cost_inr"] +
            bids["Defect_Cost"] +
            bids["Lead_Time_Penalty"]
        )

        bids = bids.sort_values(by='TCO', ascending = True).reset_index(drop=True)
        bids['Rank'] = bids.index + 1

        return bids
```

```
if __name__ == "__main__":
    try:
        raw_bids = pd.read_csv("C:/Users/Suyash Kadu/Desktop/CODING/PROJECTS/The Supplier Bid Optimization Engine (SKF)/Dataset.csv")

        TARGET_ITEM = "M12×50 Hex Bolt 8.8 Zinc (100pcs)"
        ORDER_QUANNTITY = 5000.00
        COST_PER_DEFECT = 10.00
        LEAD_TIME_PENALTY_PER_DAY = 3.00

        optimized_bids = calculate_tco(
            df = raw_bids,
            target_item = TARGET_ITEM,
            quantity = ORDER_QUANNTITY,
            cost_per_defect = COST_PER_DEFECT,
            penalty_per_day = LEAD_TIME_PENALTY_PER_DAY
        )

        print("Top 3 Optimal Supplier Split based on TCO")
        print(optimized_bids[['supplier_name', 'baseline_cost_inr', 'TCO', 'Rank']].head(3))

    except FileNotFoundError:
        print("Error: Ensure file is in same directory")
```

```
Top 3 Optimal Supplier Split based on TCO
          supplier_name  baseline_cost_inr         TCO  Rank
0   Jai Balaji Industries                380  1302585.26     1
1  Uttam Galva Steels Ltd                380  1433287.52     2
2         Nalco India Ltd                380  1465521.53     3
```

7.5. Data Validation & Algorithmic Testing

The decoupled architecture allowed for rigorous testing using a mocked CSV dataset of suppliers bidding on industrial materials like bearings.

```
    return bids

if __name__ == "__main__":
    try:
        raw_bids = pd.read_csv("C:/Users/Suyash Kadu/Desktop/CODING/PROJECTS/The Supplier Bid Optimization Engine (SKF)/Dataset.csv")

        TARGET_ITEM = "M12x50 Hex Bolt 8.8 Zinc (100pcs)"
        ORDER_QUANNTITY = 5000.00
        COST_PER_DEFECT = 10.00
        LEAD_TIME_PENALTY_PER_DAY = 3.00

        optimized_bids = calculate_tco(
            df = raw_bids,
            target_item = TARGET_ITEM,
```

7.6. Enterprise Codebase Transition

The raw Python logic (specifically the calculate_tco() function) was transitioned into an app.py script in VS Code, officially separating the analytical math from the frontend.

app.py ● config.toml

app.py > calculate_tco

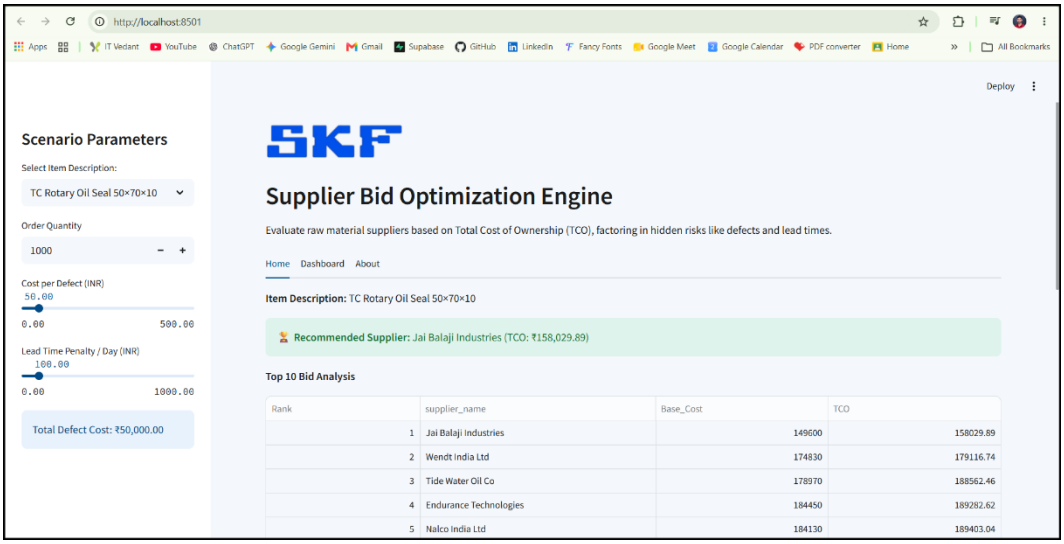
```python
import streamlit as st
import pandas as pd
import os
# ----------------------------------------
# 1. PAGE CONFIGURATION
# ----------------------------------------
st.set_page_config(page_title="Supplier Bid Optimization Engine", layout="wide")

# Safety check for the logo image
if os.path.exists("skf.png"):
    st.image("skf.png", width=200)
else:
    st.warning("Logo 'skf.png' not found. Please ensure it is in the same directory.")

st.header("Supplier Bid Optimization Engine")
st.markdown("Evaluate raw material suppliers based on Total Cost of Ownership (TCO), factoring in hidden risks like defects and lead times.")


# Initialize tabs
tab1, tab2, tab3 = st.tabs(["1. Home", "2. Dashboard", "3. About"])

# ----------------------------------------
# 2. THE CORE ENGINE (From Phase 1)
# ----------------------------------------
def calculate_tco(df, target_item, quantity, cost_per_defect, penalty_per_day):
    bids = df[df["description"] == target_item].copy()

    if bids.empty:
        return bids # Return empty dataframe if no match

    bids["Base_Cost"] = bids["unit_price_inr"] * quantity
    bids["Defect_Cost"] = (bids["defect_rate_pct"] / 100) * quantity * cost_per_defect
    bids["Lead_Time_Penalty"] = bids["lead_time_days"] * penalty_per_day

```

7.7. Interactive User Interface (UI) Architecture

The logic was wrapped in an interactive Streamlit dashboard featuring an SKF Corporate Blue theme. The interface empowers procurement managers to run dynamic "what-if" scenarios using sidebar slider inputs.

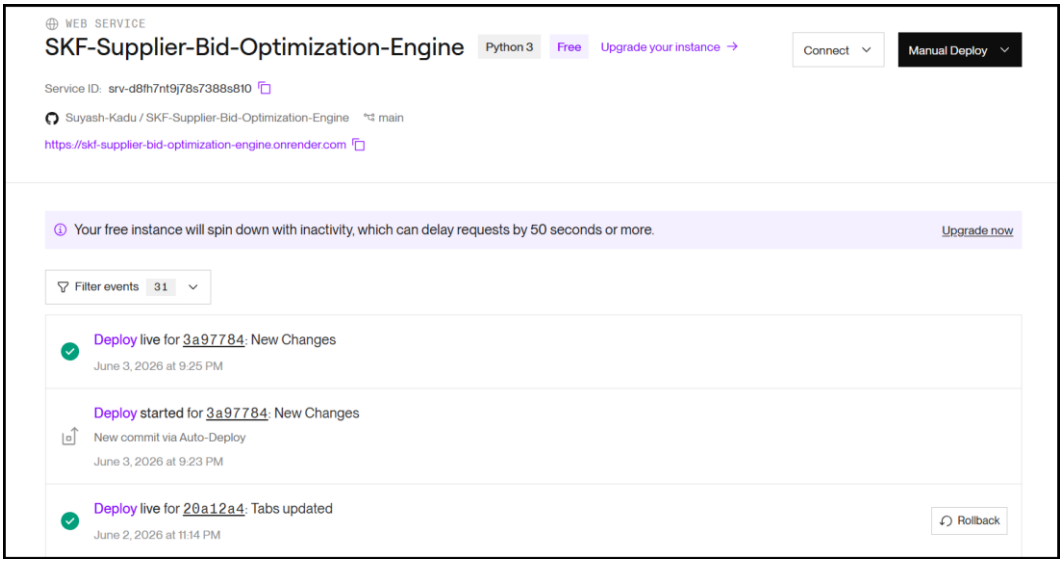
7.8. Local Environment Testing & UI Refinement

The Streamlit UI was tested on localhost using the command streamlit run app.py via the VS Code terminal.



7.9. Cloud Deployment & CI/CD Integration

The final step of Phase 1 involves deploying the application to a cloud environment (such as Render or Streamlit Community Cloud) to demonstrate production readiness and establish a continuous integration/continuous deployment (CI/CD) pipeline.
(Web Link - https://skf-supplier-bid-optimization-engine.onrender.com)



8. Phase 2: Relational Database Integration & Dynamic Visualization:-

Phase 2 elevates the application from a static prototype to a production-ready enterprise tool by replacing the mocked CSV files with a live, relational database architecture.

8.1. Database Integration

The static data structure will be migrated to Supabase (PostgreSQL). By integrating the Supabase Python client, the app will securely pull live, relational supplier data directly from the cloud database.

8.2. Caching Strategy

To maintain the lightning-fast performance established in Phase 1, Streamlit's caching mechanisms (@st.cache_data) will be heavily utilized. This prevents redundant database queries, ensuring the app doesn't refresh data every time a procurement manager adjusts a 'What-If' slider.

8.3. Dynamic Dashboarding

The Streamlit UI will be expanded into a dynamic dashboard, styled with the SKF Corporate Blue theme. It will feature dropdowns for multi-RFP filtering (e.g., dynamically filtering out irrelevant suppliers when comparing steel vs. lubricants) and interactive visualizations to compare the Base Cost versus the true bottom-line TCO.

9. Conclusion:-

The Supplier Bid Optimization Engine (SBOE) successfully translates complex supply chain variables into actionable financial decisions.

By establishing a robust mathematical backend that forces purchasing decisions to be based on Total Cost of Ownership (TCO), the tool acts as a financial lie detector for supplier bids. It proves that in global manufacturing, the lowest unit price is rarely the cheapest final cost.

Leveraging modern Python data structures (Pandas vectorization) and interactive web frameworks (Streamlit), the application provides a scalable, enterprise-grade solution. It empowers procurement managers to run dynamic "what-if" scenarios , ultimately protecting multi-million dollar manufacturing contracts from hidden financial bleed.